

Lepton MVA in ONNX

TMVA → ONNX conversion, validation, and what the scores say about our $e\mu$ selection

Chirayu Gupta · $H \rightarrow WW + \text{charm}$ · 12 August 2026

What the files turned out to be

Two TMVA weight XMLs, initially assumed to be trigger scale factors.

They are not. No HLT path, no leg p_T/η efficiency map, no data/MC ratio anywhere in them.

They are the **ttH-multilepton lepton MVA** (`mvaTTH`) — a per-lepton prompt-vs-nonprompt discriminant.

Creator / date	<code>cvico</code> , 2023-12-20
Origin	<code>CMGTools/TTHAnalysis</code> , CMSSW_12_4_12, wz-run3
Method	<code>BDT::BDTG</code> — 500 trees, MaxDepth 8, shrinkage 0.1
Classes	binary: Signal = prompt, Background = nonprompt
Transforms	<code>NTransformations="0"</code> — no input preprocessing
Training events	879,894 (muon)

Why not just read the NanoAOD branch?

Muon_mvaTTH and Electron_mvaTTH **do exist** in NanoAODv12 — verified on a central TTto2L2Nu v12 file.

We do not use them:

- the stock branch is a **Run 2** training
- these XMLs are a dedicated **2022EE** retraining
- same 13 variables, **different weights**

The stock branch would be **subtly** wrong, not obviously wrong — the worst kind. Evaluate these weights instead.

All 13 input branches verified present for **both** flavours in NanoAODv12 — 0 missing.

The 13 inputs

i	muon	electron
0–1	pt, eta	pt, eta
2–4	pfRelIso03_all, miniPFRelIso_chg, neutral remainder	same
5–6	jetNDauCharged, jetPtRelv2	same
7	jetIdx>-1 ? Jet_btagDeepFlavB[jetIdx] : 0	same
8	min(1/(1+jetRelIso), 1.5)	same
9–11	sip3d, `log	dxy
12	segmentComp	mvaIso

Isolation + jet association + impact parameters + one detector-specific variable.

Slot 7 is a **conditional cross-collection lookup**, not a plain branch.

The response formula

TMVA BoostType=Grad, two classes:

```
raw    = sum_t res_t(x)           # leaf 'res' values summed, no extra factor
score = 2 / (1 + exp(-2*raw)) - 1 # in (-1, 1)
```

Shrinkage is already folded into the stored leaf values. Do not reapply it.

Verified empirically rather than assumed:

	leaf <code>res</code> absmax
tree 0	0.0999 $\approx$ shrinkage 0.1
tree 499	0.0224

The cap at ~ 0.1 and the decay across the ensemble are exactly what gradient boosting with pre-folded shrinkage produces.

Mapping TMVA trees onto ONNX

Node attributes in the XML:

attribute	meaning
IVar	feature index (-1 on leaves)
Cut	split threshold
nType	-99 marks a leaf
pos	s / l / r — root, left, right
res	leaf response

TMVA sends `var <= Cut` to the **left** child.

ONNX `BRANCH_LEQ` has the same convention → `true_branch = left`.

Emitted as a `TreeEnsembleRegressor` (raw sum) followed by the closed-form squashing, so the graph reproduces the **full score**, not just the raw sum.

Validation: three implementations

1. ONNX

`TreeEnsembleRegressor` + squashing

2. numpy

independent recursive mask-based tree walk

3. `TMVA::Reader`

the reference implementation, via LCG view

`LCG_104a/x86_64-el9-gcc12-opt`

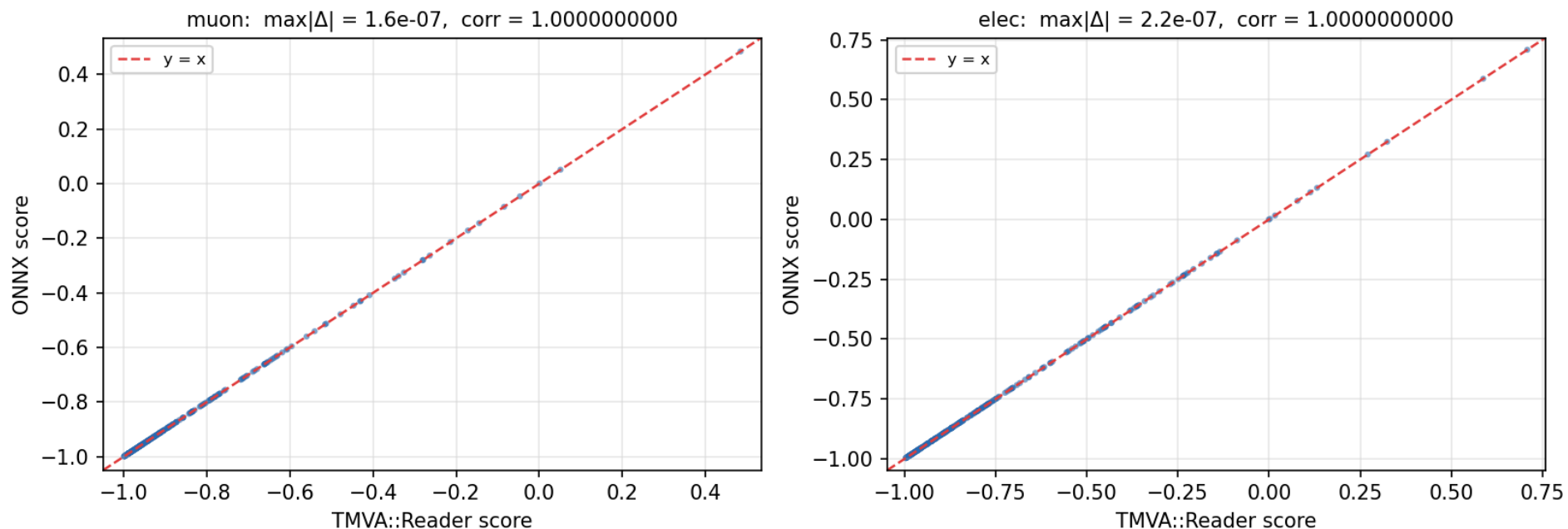
`TMVA::Reader` is slow (scalar API, 500 trees): 2000 points times out at 2 min;
300 points suffices.

ONNX-vs-numpy alone is **not sufficient** —
both encode *my* reading of the
format.

Only `TMVA::Reader` can catch an inverted
branch or a double-applied
shrinkage.

Validation result

ONNX vs TMVA::Reader — identical to float32 round-off



pair	N	max $ \Delta $	corr
ONNX vs numpy (muon / elec)	2000	2.2e-07 / 2.9e-07	—
ONNX vs TMVA::Reader (muon)	300	1.6e-07	1.0000000000
ONNX vs TMVA::Reader (elec)	300	2.2e-07	1.0000000000

Test suite — `test_leptonmva.py`

	test	what it guards against
T1	ONNX vs <code>TMVA::Reader</code>	misread format — the authoritative check
T2	ONNX vs independent numpy walk	ONNX emitter bugs
T3	score finite and in $(-1, 1)$	wrong squashing; $\pm 1e6$ inputs probed
T4	500 trees / 13 features vs XML	truncated or dropped trees
T5	determinism on repeat	nondeterministic kernels
T6	single-row == full batch	cross-row leakage
T7	<code>jetIdx=-1</code> → btag 0, slot 7 influential	the guard, and a vacuous test
T8	float64 input rejected	silent dtype coercion

20 / 20 PASS. T7 also confirms slot 7 matters (max shift 1.40 muon, 0.55 electron)
— so the `jetIdx` guard is not cosmetic.

So: should we use it?

The conversion succeeded. That does not by itself mean the cut belongs in our analysis.

The deciding question:

How much nonprompt background is actually in our selection?

Measured directly — central `TTto2L2Nu` v12, 400k events, with the `hww_combine_2dcat` lepton selection reproduced:

- muon: tight ID + tight PF iso, $p_T > 10$, $|\eta| < 2.4$
- electron: wp80iso, $p_T > 10$, $|\eta| < 2.5$
- exactly one of each, opposite sign, lead $p_T > 20$, sublead > 10

49,070 $e\mu$ events survive. Truth label from `genPartFlav`.

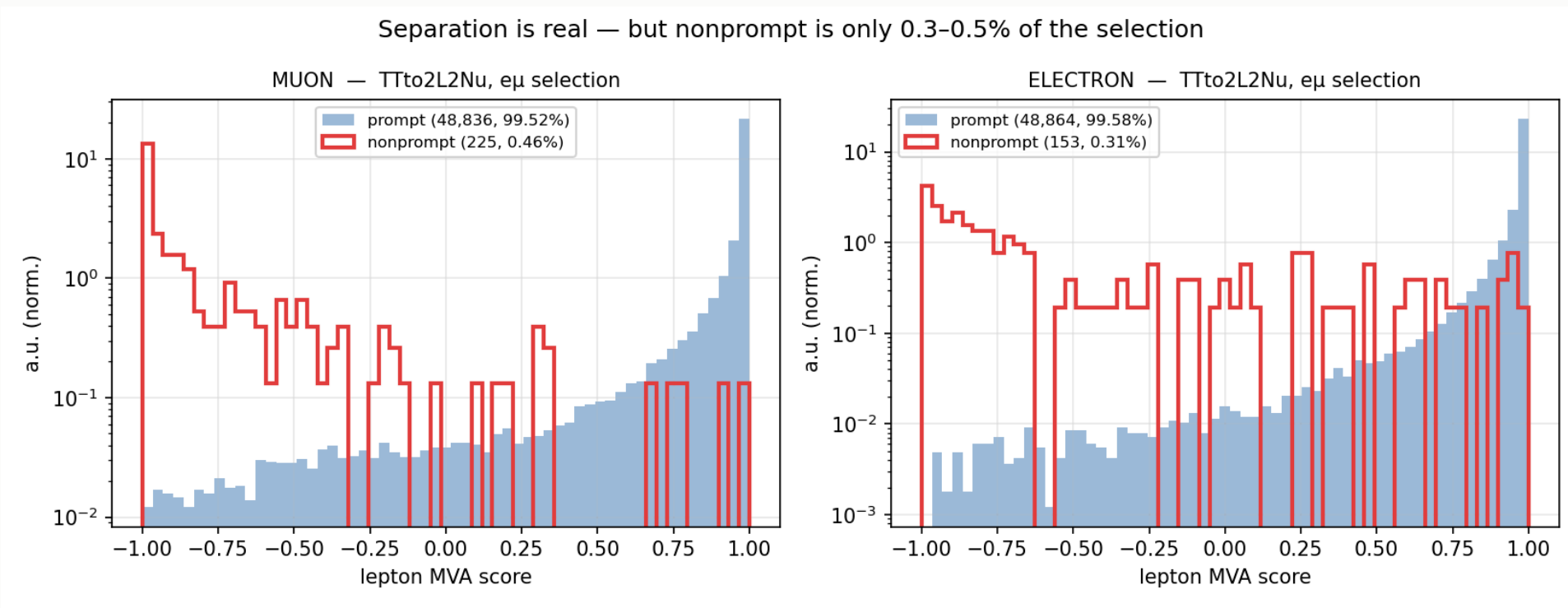
The headline number

flavour	prompt	nonprompt	other
muon	48,836 — 99.52%	225 — 0.46%	9 — 0.02%
electron	48,864 — 99.58%	153 — 0.31%	53 — 0.11%

Nonprompt leptons are 0.3–0.5% of the selection. The existing cut-based tight ID + isolation has already removed essentially all of them.

The MVA is aimed at a background we have already eliminated by other means.

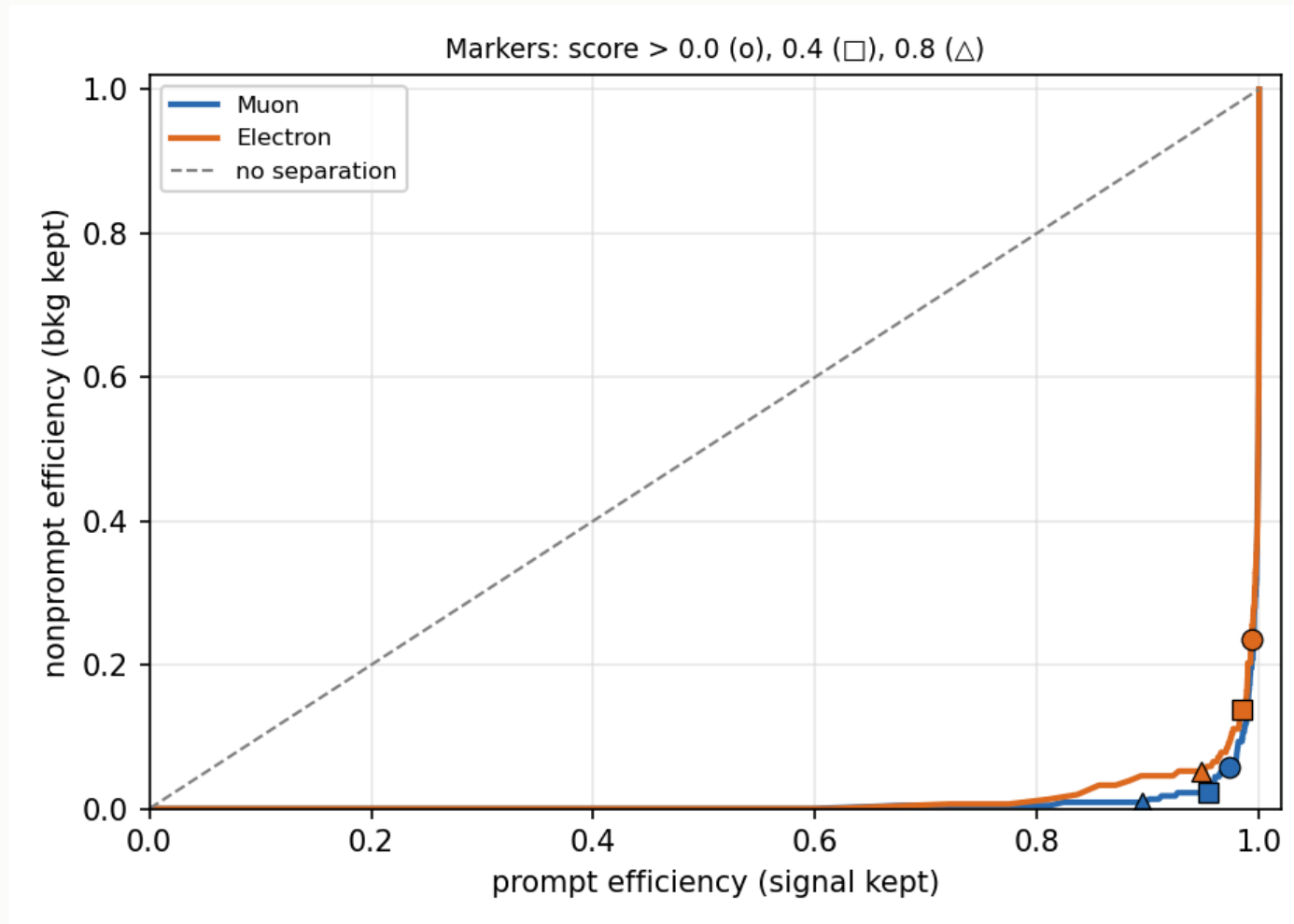
The separation itself is real



	muon	electron
median score, prompt	+0.9952	+0.9945
median score, nonprompt	-0.9510	-0.7640

Electron separation is weaker — `mvaIso` sits in slot 12 and already overlaps heavily with the `wp80iso` cut applied upstream.

What a cut would cost



Muon working points:

cut	eff(prompt)	eff(nonprompt)	rejection
-----	-------------	----------------	-----------

The trade, quantified

Taking `score > 0.4` on both legs:

removes	~98% of an already-0.46% contamination → ~0.45 pp
costs	~4.5% of prompt muons + ~1.5% of prompt electrons → ~6% of signal

~6% of signal efficiency to remove ~0.45% of background.

The loss lands directly on H+c signal — whose limit is already dominated by MC statistics (autoMCStats = -255 of the -523 total systematic budget).

No reference analysis of ours does this

analysis	lepton treatment
AN-23-102 (our Run 2 predecessor)	no lepton MVA
HH $\rightarrow$ bbWW Run 3 (same $e\mu$, 13.6 TeV)	cut-based ID + PF rel-iso
ttH-multilepton (2011.03652)	source of these weights — "tight lepton" = loose + MVA cut

ttH-multilepton works in **same-sign 2ℓ / 3ℓ / 4ℓ** , where nonprompt is a *leading* background and there is no OS continuum.

Ours is **OS $e\mu$, 82% real $t\bar{t}$** , both leptons genuinely prompt.

The technique is imported from a different topology — the 0.4% measurement is the quantitative version of that observation.

The calibration cost, if we did adopt it

From the ttH paper's efficiency section — once "tight" is defined by an MVA cut:

1. measure efficiency by **tag-and-probe in $Z \rightarrow \ell\ell$**
2. cross-check in an **OS $e\mu + \geq 2$ jets $t\bar{t}$ CR**, nonprompt subtracted via an SS $e\mu$ sideband
3. take the DY-vs- $t\bar{t}$ difference as a **1–2% systematic**

That $t\bar{t}$ control region **is our signal region**.

And SFs for these specific 2022EE weights are unlikely to exist publicly.

Plus a full reprocessing campaign.

Recommendation

Do not adopt a lepton-MVA cut in the $e\mu$ signal region. The cost/benefit is clearly negative on these numbers.

Where the converted models could still earn their keep:

use	note
Nonprompt/QCD control region	<i>invert</i> the MVA → clean enriched region. Much better use than cutting in the SR.
Input feature, not a cut	costs no signal efficiency by construction; needs retraining, and gain is small at 0.4%
Evidence the selection is already clean	this measurement itself — a publishable statement

Caveats

- **Single TT file**, 400k events → 225 nonprompt muons, 153 electrons.
Adequate for a 0.4%-vs-6% decision; too coarse for a precise WP scan.
- **$t\bar{t}$ only**. V+jets/QCD have higher nonprompt fractions, but they are 7.3% and ~0% of the SR — the conclusion is not sensitive to this.
- **$\log|d_{xy}|$ floor unresolved**. Used $1e-6$; the training producer's convention is unknown. Affects few leptons, but must be pinned before any production use.

Wiring plan — PROPOSAL, not implemented

No production code has been modified. This is the plan someone would follow.

Template to copy: the existing **negrw** integration —

`analysis/processors/base.py:142-147` plus the `negrw:` block in the workflow YAML.

Same shape: config-gated, lazily-loaded model, extra dumped columns.

Six steps. Steps 1–2 are the ones that break a condor submission if skipped.

#	step
1	model files on AFS  done
2	<code>leptonmva:</code> config block in the workflow YAML
3	per-lepton model routing by <code>pdgId</code>
4	feature construction — 13 columns, XML order
5	dump as columns, apply no cut
6	pre-flight checks

Step 1 — models on AFS (already done)

The trap that would break the first submission. Your own `negrw` config says it:

must be on AFS, not EOS — the worker singularity container cannot read `/eos/user` paths (PermissionError), but it reads AFS fine.

The converted models were written to **EOS only**.

Already staged, mirroring `negrw_model/`:

```
/afs/cern.ch/user/c/cgupta/leptonmva_model/  
muon_mvATTH_2022EE.onnx      0.38 MB  
electron_mvATTH_2022EE.onnx  0.40 MB
```

Master copies stay on EOS. Note AFS quota is at **91%** — 0.8 MB is negligible, but it is close to the limit.

Step 2 — the config block

Added to the workflow YAML alongside `negrw:`, so nothing else is affected:

```
# AFS paths -- workers cannot read /eos/user (see the negrw note).  
# Master copies live in /eos/user/c/cgupta/HToWW/leptonmva/onnx/  
leptonmva:  
  model_dir: /afs/cern.ch/user/c/cgupta/leptonmva_model  
  muon_model: muon_mvaTTH_2022EE.onnx  
  electron_model: electron_mvaTTH_2022EE.onnx  
  dxy_floor: 1.0e-6      # OPEN -- training convention unknown
```

Differs from `negrw`: no datasets: gating. All 13 inputs are reco-level, so this runs on **data as well as MC**. `negrw` is gen-level and therefore MC-only.

Step 3 — per-lepton model routing

`objects["ll_pair"].l1 / .l2` are ordered by **p_T , not flavour.**

`l1` is the muon in some events and the electron in others.

Route on `abs(pdgId)` — **13** → **muon model**, **11** → **electron model**:

```
is_mu_l1 = abs(ll.l1.pdgId) == 13
```

The selection guarantees exactly one muon and one electron per event (`one_muon_one_electron`), so the clean implementation is:

1. score the muon collection with the muon model
2. score the electron collection with the electron model
3. map each back into its `l1 / l2` slot

Scoring `l1` with a single model would silently mix the two BDTs.

Step 4 — feature construction

13 columns in **XML order** (slide 3). Cached session, lazy-loaded exactly like `self._negrw_bundle`:

```
if self._leptonmva_sess is None:                # once per worker
    self._leptonmva_sess = ort.InferenceSession(cfg["muon_model"], ...)

idx = lep.jetIdx
safe = ak.where(idx > -1, idx, 0)                # mask BEFORE indexing
btag = ak.where(idx > -1, events.Jet.btagDeepFlavB[safe], 0.0)
...
X = np.column_stack([...]).astype(np.float32)   # float32 or it raises
s = ak.unflatten(sess.run(None, {"X": X})[0].ravel(), ak.num(lep.pt))
```

trap	consequence if missed
index without the <code>-1</code> guard	silently reads the last jet in the event
<code>InferenceSession</code> per chunk	dominates runtime
float64 passed in	raises (T8)
<code>log dxy </code> when <code>dxy == 0</code>	<code>-inf</code> — floor convention still open

Steps 5 & 6 — output and pre-flight

5 · Dump columns, apply no cut

```
variables_map["lep_mva_l1"] = score_l1  
variables_map["lep_mva_l2"] = score_l2
```

Keeps the wiring **use-agnostic** — cut vs. CR vs. input feature is still undecided, and the measurement says do not cut in the SR. Columns cost nothing and let the decision be made later from the parquet.

6 · Pre-flight

1. confirm `onnxruntime` is in the **condor worker** env (1.17.3 in `b_hive` interactively — the worker env must be checked, not assumed)
2. run `test_leptonmva.py` → expect 20/20
3. one-file local run, then compare a handful of scores against the standalone diagnostic
4. resolve the `dxy` floor before any *physics* use of the scores

Artifacts — all durable on EOS

```
/eos/user/c/cgupta/HToWW/leptonmva/  
onnx/muon_mvaTTH_2022EE.onnx      0.38 MB  <- validated  
onnx/electron_mvaTTH_2022EE.onnx  0.40 MB  <- validated  
tmva_to_onnx.py                   converter (checks Options, refuses transforms)  
numpy_ref.py                      independent numpy evaluator  
tmva_reference.py                 TMVA::Reader reference (needs LCG view)  
validate_vs_tmva.py              the comparison  
tmva_reference.npz               frozen reference inputs + scores  
diagnostic_nonprompt.py          the ep measurement  
test_leptonmva.py               the 20-test suite (T1-T8)  
  
/afs/cern.ch/user/c/cgupta/leptonmva_model/    <- worker-readable copies  
muon_mvaTTH_2022EE.onnx, electron_mvaTTH_2022EE.onnx
```

Re-run the full suite at any time:

```
cd /eos/user/c/cgupta/HToWW/leptonmva  
$MAMBA_EXE run -n b_hive python3 test_leptonmva.py    # expect 20/20
```

No production code was modified

Summary

Conversion: exact — corr = 1.0000000000 vs `TMVA::Reader`, 20/20 tests pass

Nonprompt in our $e\mu$ selection: 0.3–0.5%

A cut would cost ~6% of signal to remove ~0.45% of background

Recommend: do not cut. Keep the models for a nonprompt CR.

Wiring plan is ready to execute if wanted — models already staged on AFS, columns-only, no cut. Open item: the `log|dxy|` floor convention.